

# Formal Agent Kernel - Validation Engine for llms

Saurav Jaiswal

## 1 Problem Statement

### 1.1 Overview

Modern AI agent systems and frameworks (such as LangChain, LangGraph, AutoGen, and CrewAI) are increasingly capable of autonomous reasoning, multi-step planning, and tool execution. These systems are rapidly being integrated into sensitive, real-world workflows, spanning software deployment, autonomous vehicles, healthcare, and financial transaction processing. However, a critical vulnerability exists: these frameworks currently operate under assumed correctness. Large Language Models (LLMs) are inherently probabilistic; they can hallucinate, drift from their intended system prompts, and fall victim to malicious prompt injections or adversarial environmental perturbations. Existing mitigation methods—such as heuristic guardrails, natural-language prompt instructions, and static output filters—operate primarily at the text or policy level. Because these policies are written in ambiguous natural language, they map poorly to reliable, machine-checkable rules. Consequently, current systems offer no provable, formal semantics to guarantee that an agent’s actions will safely remain within predefined operational boundaries before execution.

### 1.2 Research Problem

In distributed, autonomous, and regulated environments (e.g., enterprise banking), it is highly dangerous to rely on an AI model to both reason about an action and execute it without an independent validation layer. A major architectural gap exists where probabilistic models generate actions that are syntactically valid (e.g., a properly formatted SQL query or API payload) but semantically incorrect or unsafe regarding the agent’s actual motivation, role, or corporate policy. The Core Problem: There is currently no formal, framework-agnostic runtime mechanism that deterministically guarantees an AI agent’s executed actions are consistent with its declared motivation, role, and constraints independent of the underlying probabilistic AI model.

## 2 Proposed Approach

This project proposes the development of the Formal Agent Kernel (FAK)—a declarative runtime validation system that enforces formally specified behavioral contracts on AI agents before execution. The foundational paradigm of FAK is the strict decoupling of reasoning from execution. Agents may reason freely and propose structured actions, but only the FAK can authorize and execute them. FAK operates as a deterministic execution gatekeeper that translates ambiguous policies into rigorous mathematical logic. Instead of parsing unstructured natural language, FAK enforces formally declared contracts over structured actions (e.g., JSON payloads). Each agent and its proposed interaction is modeled using a rigorous formal tuple:  $\text{Agent} = \langle \text{Motivation}, \text{GoalSpace}, \text{ActionSpace}, \text{Constraints} \rangle$ .

At runtime, when an agent proposes an action, the FAK computes a logical proof:

$$\langle \text{Agent}, \text{Goal}, \text{Action}, \text{Context} \rangle \models \text{Constraints}$$

Based on this evaluation, the FAK’s Decision Engine outputs one of several enterprise-ready verdicts:

- **ALLOW:** The action is provably safe and proceeds to execution.

- **DENY**: The action violates a contract and is blocked.
- **REQUIRE\_APPROVAL**: The action requires human-in-the-loop inspection (e.g., sending a push notification for a high-risk trade).
- **REQUIRE\_CONFIRMATION / SELF\_CORRECT**: The action is blocked, and a machine-checked rationale is returned to the agent, forcing it to self-examine and replan safely.

### 3 System Architecture

The FAK is designed to be runtime-agnostic, domain-agnostic, and framework-agnostic. It plugs seamlessly into orchestration layers like LangGraph as a middleware node, forming a Directed Acyclic Graph (DAG) pipeline:  $\text{AgentNode} \rightarrow \text{FAKNode} \rightarrow \text{ExecutionNode}$ .

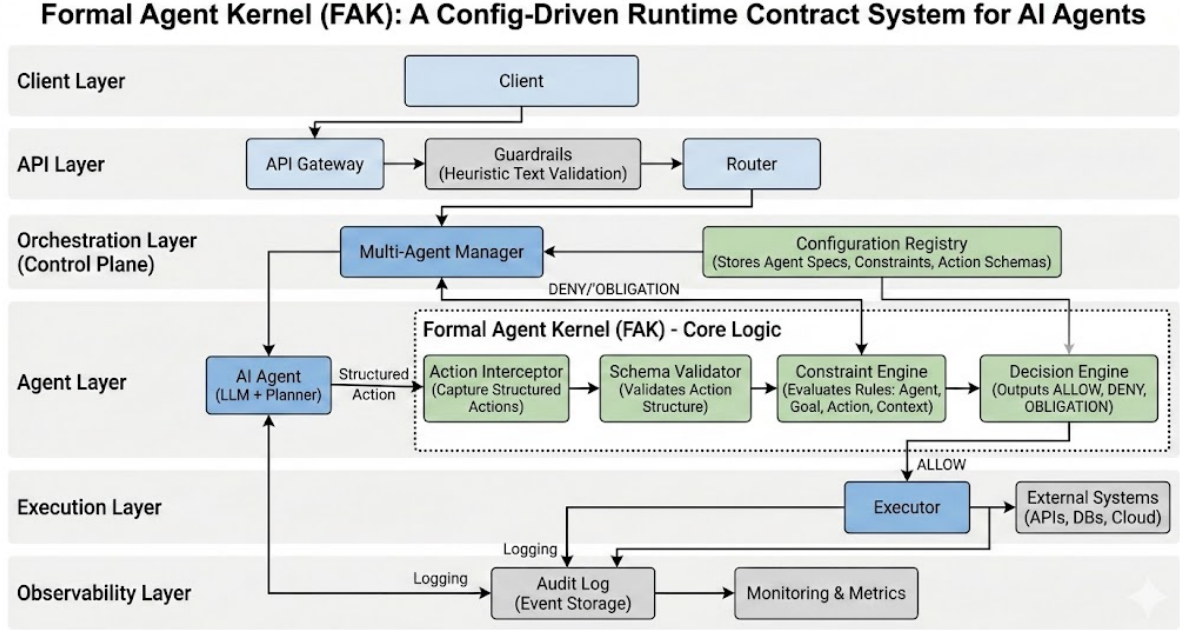


Figure 1: Architecture

#### 3.1 Core Component

- **Action Interceptor & Schema Validator**: Hooks into the agent’s decision pipeline, catching the proposed JSON action and mathematically guaranteeing it conforms to expected schemas.
- **State Tracker / Context Engine**: Maintains continuous system state. To avoid state-space explosion, it can utilize ultra-fast Top-k retrieval to fetch only the most semantically relevant constraints for the proposed action.
- **Constraint Engine (Policy Verifier)**: Evaluates the structured payload against declarative Boolean predicates. It performs sequent safety checks  $\Gamma \vdash \psi$  to prove that the current state entails the safety obligation.
- **Decision Engine (Referee)**: Synthesizes the constraint evaluation to issue an **ALLOW**, **DENY**, or **OBLIGATION** response, along with an auditable rationale.
- **Audit Logger**: Writes every verdict and mathematical proof into an immutable state store for regulatory compliance

### 3.2 Config Driven Design

A primary design principle of FAK is that the core Python logic remains static; behavior is entirely defined via configuration. FAK does not contain hardcoded finance or DevOps logic. Instead, deployers define safety boundaries using three primary YAML files.

- `agent_specs.yml` - Defines the Actor and Intent
- `action_schema.yml` - Defines the operation
- `constraints.yml` - Defines the declarative Contracts

## 4 Novelty or Innovation

- While basic execution guardrails exist (e.g., Open Policy Agent (OPA) or heuristic filters), FAK introduces several key innovations specific to multi-agent generative AI: Semantic Intent Verification: Existing frameworks focus strictly on checking raw API parameters. FAK uniquely incorporates abstract Motivation and Goal Space into its mathematical validation tuple  $\langle M, G, A \rangle \models C$ . It verifies whether an action aligns with an agent’s abstract intent, shifting the paradigm from basic data filtering to semantic intent verification.
- Universal Config-Driven Execution: By restricting FAK to operate purely on three generic abstractions—Actor (Agent), Intent (Goal), and Operation (Action)—via YAML configs, FAK achieves true plug-and-play middleware status. It can be installed via `pip install fak-kernel` and dropped into LangGraph or AutoGen without rewriting backend logic.
- Dynamic Feedback Loop (Self-Correction): Unlike stateless IT engines that just drop an invalid request (causing an LLM to hallucinate or crash), FAK generates a machine-checked rationale upon denial. This rationale is returned to the agent, triggering a structured replanning or self-examination phase.
- Millisecond Performance via Top-k Retrieval: Formal verification traditionally suffers from state-space explosion. FAK circumvents this by using  $O(\log m)$  embedding indexes to retrieve only the Top-k constraints relevant to a specific action, reducing the verification overhead to mere milliseconds.

## 5 Research questions

Primary Question: How can we formally enforce runtime behavioral contracts for AI agents in a configuration-driven manner, guaranteeing that an agent’s actions are consistent with its declared motivation and constraints, independent of the underlying probabilistic AI model? Secondary Questions: How can we maintain millisecond latency in formal verification during real-time, high-frequency multi-agent interactions? How can deterministic constraints handle the non-deterministic output of LLMs effectively without causing infinite replanning loops?

## 6 Tech Stack

The technology stack is designed for modularity, low-latency execution, and seamless integration with modern Python-based AI orchestration frameworks.

Table 1: Placeholder Caption

| Layer                    | Technology / Framework                                                                       |
|--------------------------|----------------------------------------------------------------------------------------------|
| API / Routing Layer      | GoRoutines/SpringBoot/FastAPI                                                                |
| Core Kernel              | Rust/Golang/Python 3.10+                                                                     |
| Agent Orchestration      | LangGraph / LangChain (for DAG workflow integration)                                         |
| Rule / Constraint Engine | Custom DSL Parser (e.g., ANTLR4) or Expression Evaluator (CEL)                               |
| Configuration Format     | YAML (for <code>agent_specs</code> , <code>constraints</code> , <code>action_schema</code> ) |
| State & Threat Tracking  | In-memory Vector Store (for Top- $k$ rule retrieval) / Redis                                 |
| Storage / Database       | PostgreSQL / SQLite (for configurations and state mapping)                                   |
| Observability / Audit    | Elasticsearch (optional) / Structured JSON Logging                                           |

## 7 Timeline

The project will be executed over a 12-week schedule, categorized into Research, Core Implementation, Integration, Evaluation, and Writing phases

Table 2: Placeholder Caption

| Phase   | Core Focus               | Key Dates         | Duration  |
|---------|--------------------------|-------------------|-----------|
| Phase 1 | Research & Proposal      | 1 Feb – 9 Mar     | 5.5 Weeks |
| Phase 2 | Core Development         | 10 Mar – 25 May   | 11 Weeks  |
| Phase 3 | Multi-Agent & Evaluation | 26 May – 13 July  | 7 Weeks   |
| Phase 4 | Interim Submission       | 14 July – 16 July | 3 Days    |
| Phase 5 | Refinement & Writing     | 17 July – 31 Aug  | 6.5 Weeks |
| Phase 6 | Final Buffer             | 1 Sept – 8 Sept   | 1 Week    |

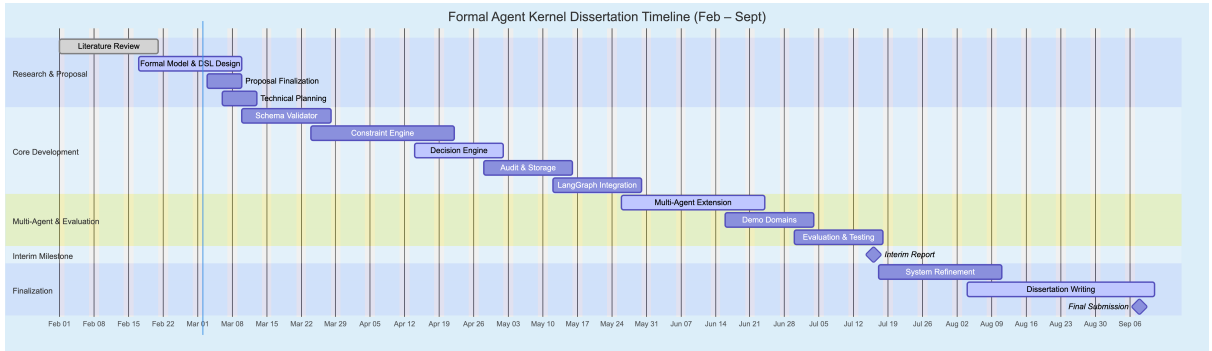


Figure 2: Gantt

## 8 Expected Outcome

The successful completion of this project will yield the following tangible and theoretical outcomes:

- **A Deployable, Reusable Middleware Package:** A lightweight, runtime contract enforcement system (installable as a package) that cleanly separates agent reasoning from execution authority.
- **Deterministic Safety for Enterprise AI:** By compiling declarative YAML constraints into formal logic, the FAK will guarantee that no unauthorized actions reach external APIs or databases. It provides a mathematical straightjacket for probabilistic AI models.
- **High-Performance Verification:** Empirical validation showing that the FAK incurs negligible runtime overhead (in the order of 1 to 5 milliseconds per action), proving that formal verification can scale safely alongside high-frequency AI applications.
- **Absolute Auditability:** An immutable audit log system containing the explicit formal proof for every ALLOW, DENY, or REQUIRE\_APPROVAL verdict generated by the AI system. This

creates a compliance-ready environment for highly regulated industries like finance, healthcare, and infrastructure management.

- Cross-Domain Portability: Demonstration of the FAK securing disparate agent domains (e.g., a Database Query Agent vs. a DevOps Deployment Agent) simply by swapping the YAML configuration files, without altering the Python kernel logic